

Thesis Defense Script

Full Script — All 29 Slides, Sections 1–6 — FINAL PRODUCTION VERSION

GOLD = say first — connecting words, transitions, bridge into the idea.

NAVY = main point / slide content.

Section 1 — Introduction

Slide 1 — Cover Page

Good morning, respected Chairman, Reviewers, Committee Members, and distinguished guests. My name is PHORN Sreypov, and today I'm honored to present my thesis, “Electrical Load Forecasting and Demand Management Using Meta-Stacking Ensemble Learning with a Smart Forecasting Platform.” This research was conducted during my internship at the EDC–AFD–EU Platform for Research and Training on Power System, under the supervision of Mr. NOP Phorn Nara, with academic guidance from my advisor, Dr. HAS Sothea.

Speaker note: Memorize this slide closely — it's the only one read almost verbatim. Maintain direct eye contact with the committee, not the slide presentation.

Slide 2 — Table of Contents

Before going further, let me give you a quick map of where this presentation is going. I'll walk through six parts: the introduction to my internship and host organization, the project description and the problem that motivated this work, a review of related literature, the methodology I developed, the results and discussion, and finally the conclusion together with a short demonstration of the platform.

Speaker note: Say this fast and light — orientation, not content. Don't read the numbered bullets “01, 02, 03...” directly off the slide.

Slide 3 — Internship Overview

Let's start with where this work took place. I completed a four-month hybrid internship, combining on-site and remote work, at the Host Platform for Research and Training on Power System, working as a Data Scientist. My field centered on electricity load forecasting & management, focusing specifically on ML and DL modeling using real-world operational power data from EDC.

Slide 4 — EDC-AFD-EU Project Overview

This internship sits inside a larger initiative, so it's worth understanding that platform before I describe the specific problem I focused on. The Platform for Research & Training on Power System of the EDC-AFD-EU Project was established by EDC under the AFD-funded Grid Modernization Project to strengthen Cambodia's power-sector technical capacity. It is jointly implemented by Electricité du Cambodge, the Institute of Technology of Cambodia, and

Université Grenoble Alpes. Our mission is to develop highly qualified human resources through education, research, and international collaboration at the Master's and PhD levels.

Section 2 — Project Description

Slide 5 — Project Introduction

With that context in mind, let's look at the problem this project was actually built to solve.

This project develops a Meta-Stacking Ensemble machine learning framework and an interactive forecasting platform to support EDC amid Cambodia's rapidly growing power demand. From 2010 to 2024, Cambodia's installed power capacity grew nearly 9x, from approximately 580 megawatts to 5,044 megawatts, while energy delivered rose nearly 8x, from 2,500 gigawatt-hours to 19,419 gigawatt-hours, with growth visibly accelerating after 2019. Official projections show demand could reach 66 terawatt-hours by 2040, more than three times today's level.

Speaker note: This is a 'show the concern' slide — state the growth numbers with significant weight, as they form your core justification for urgency.

Slide 6 — Problem Statement

Despite this massive growth, the way EDC actually forecasts demand hasn't kept pace.

Cambodia's electricity demand grew approximately 80% from 2019 to 2024, but EDC still forecasts manually, limiting their ability to capture complex, non-linear demand patterns. This manual workflow creates three core vulnerabilities: first, an inconsistent data environment where manual cleaning introduces silent errors; second, a fragmented workflow where data, models, and results are disconnected; and third, no accessible tools or interface for operational staff to easily run automated forecasts.

Slide 7 — Project Objectives

That need is exactly what shaped the goal and objectives of this research. The primary objective is to develop a Meta-Stacking Ensemble framework for short-term load forecasting on Cambodia's complex grid and operationalize it as an accessible web platform. Our project scope maps out four targets: first, identify the most influential input variables using four multiple methods; second, compare feature groups to determine the best-performing set; third, build and validate ML models with rolling-year validation; and fourth, deliver an interactive platform for non-technical users. The operational scope is restricted to 48-hour operational load forecasting, while real-time SCADA and automated controls are out of scope.

Speaker note: Don't read the objectives like a dry list. Say "First... Second... Third... And fourth..." as a continuous sentence with your eyes on the committee.

Slide 8 — Project Planning

Before going into the technical details, let me quickly show you how this work was actually planned and executed. Over a 16-week timeline, I ran parallel development workstreams. The ML Research Pipeline moved from data collection, feature engineering, and

feature selection through model tuning, rolling validation, and stacking. In parallel, the Web Platform Development advanced through system architecture, backend implementation, frontend implementation, and production deployment. Our actual timeline tracked closely to the plan, serving as an efficient phase-gate development lifecycle.

Section 3 — Literature Review

Slide 9 — Literature Review

Before designing the methodology, it's worth grounding it in what's already been done in the field — so let's look at the literature. I benchmarked five key studies. Nara, Has, & Bun (2026) developed an hourly STLF model for the Cambodia EDC grid using a stacking meta-ensemble via linear regression, achieving a 3.86% MAPE, but left a gap with no automated feature selection and limited weather data. Yaprakdal & Arisoy (2023) introduced a multi-stage filter-wrapper feature pipeline yielding a 1.2% MAPE reduction, but omitted an ensemble stacking approach. Vianita & Tantyoko (2025) benchmarked distinct models on tropical climate datasets proving no single model dominates, but ran models in isolation. Taheri (2025) combined tree-based ensembles via a secondary meta-model but assumed predefined feature subsets. Finally, Beloev (2025) proved that accounting for operational telemetry delays prevents up to 15% artificial data leakage.

Speaker note: Use this spoken narrative as your summary while the table sits behind you as supporting technical detail. Do not read the table cell by cell.

Section 4 — Methodology

Slide 10 — Overall Methodology

That gap in the literature is exactly what shaped the methodology I built — let me walk you through the overall flow. The project is divided into two continuous stages. Stage 1 is the ML Forecasting Pipeline, moving from load and weather data collection, through data cleaning and feature engineering, into a three-stage feature selection funnel. This flows into a 4-fold expanding rolling validation window, model development, and grid search hyperparameter tuning. The optimized pipeline directly links to Stage 2, System Development, where the framework is operationalized using a decoupled React frontend, FastAPI backend, and PostgreSQL relational database.

Slide 11 — Data Collection

Let's start at the foundation — the data itself. Data was provided under the EDC Grid Modernization Project. Our target variable is the aggregate national grid system load spanning 2019 to 2024. To extend the single-location correlation baseline from prior work, we integrated a five-location spatial weather matrix from Open-Meteo covering Phnom Penh city, Sihanoukville province, Siem Reap, Ratanakiri, and Svay Rieng. This provides a multi-location weather profile to feed our feature framework.

Slide 12 — Data Cleaning and Feature Engineering

Raw data at this scale is never clean, so the next step was preparing it properly. Data cleaning combined domain rules, Seasonal Hybrid ESD, Isolation Forest, and Local Outlier Factor to handle anomaly detection, with time-series-consistent interpolation for missing values. Because Open-Meteo's historical archive ends in March 2021, we merged observed data with forecasted data through December 2024 to create a continuous, 52,608 x 1 target matrix. Our feature engineering generated 3 lag features, 12 rolling statistics, 15 calendar and cyclical encodings using sine/cosine transformations, 6 momentum features, and a working-day indicator, expanding the raw columns into a high-dimensional feature space.

Slide 13 — Feature Selection Framework

This next part is the core methodological contribution of this thesis — a multi-stage framework designed to systematically filter features. Stage 1 applies statistical filtering by combining Pearson correlation and Mutual Information independently to build a union candidate pool of 66 features. Stage 2 executes an advanced model-based feature importance analysis, combining Permutation Importance and SHAP attribution values across three base architectures: Random Forest, XGBoost, and MLP. This establishes a consensus ranking and model agreement criterion. Stage 3 uses category sensitivity testing across feature groups to select our final optimized feature subset for our direct 48-hour operational forecast.

Speaker note: Speak with high confidence here — this is your unique academic contribution.

Slide 14 — Rolling-Year Validation

To ensure both our feature selection and our models generalize safely without temporal data leakage, we implemented a rolling-year validation scheme. We designed a 4-fold expanding window: Fold 1 trains on 2019 and tests on 2020; Fold 2 expands to train on 2019–2020 and tests on 2021; Fold 3 expands through 2021 to test on 2022; and Fold 4 trains on 2019–2022 to test on 2023. Crucially, the full 2024 calendar year was reserved as a final holdout dataset, kept completely untouched until all feature selection, hyperparameter tuning, and meta-stacking loops were fully locked in.

Slide 15 — Model Development and Hyperparameter Tuning

With the validation scheme fixed, the next step was building the multi-level architecture. We trained four base algorithms: Random Forest as a bagging ensemble, XGBoost as boosting trees, SVR as our kernel method, and a Multi-Layer Perceptron neural network. Their out-of-fold predictions from 2020 to 2023 were compiled into a meta-dataset to train two distinct secondary meta-learners: Meta-Linear Regression and Meta-SVR. Every base model and meta-learner was tuned via exhaustive grid search on the winning feature group, selected by the best mean MAPE across all four expanding folds, before running a single evaluation check against the untouched 2024 holdout.

Slide 16 — Model Phase Summary

Let me bring all of that together in one summary view before moving to the system itself.

This architecture summary shows our full engineering progression: raw data cleaning and feature engineering build our selection set, which is split into our 4 expanding validation folds. Parallel hyperparameter tuning optimizes our base models to generate out-of-fold predictions. These predictions create our 4-variable Meta-Stacked dataset from 2021 to 2024, which trains our Linear Regression and SVR meta-learners for final 2024 holdout evaluation.

Slide 17 — System Architecture and Development

With the forecasting engine finalized, the next step was turning it into something operational for EDC staff.

I designed the platform across four clean layers: a Presentation Layer built using Vite React, TypeScript, and TailwindCSS for role-based views; an API Gateway Layer using FastAPI with Python 3.11, Nginx, and secure JWT Authentication; an ML Intelligence Layer managing feature engineering and real-time inference via a scikit-learn model registry; and a Data Persistence Layer utilizing PostgreSQL with async SQLAlchemy ORM mapping. Our automated data pipeline handles the complete lifecycle: XLSX/CSV Upload -> 5-Stage QA Pipeline -> In-Memory Feature Engineering -> Model Inference -> Interactive Forecast Output.

Slide 18 — System Development Stages & QA

Building this system required a disciplined full-stack engineering approach. Development moved through four planned stages: Foundation DevOps, Core ML Engine assembly, API integration, and final QA Delivery. The full application is containerized with Docker and managed via a GitHub Actions CI/CD pipeline, with our 14-phase build plan currently 93% complete. The system features an active model registry lifecycle and is fully verified through 87 automated unit tests hitting an 89% code coverage mark, backed by 26 automated end-to-end browser tests.

Section 5 — Results and Discussion

Slide 19 — Feature Selection Analysis (Baselines)

Let's start with how well our feature selection framework performed against standard baseline methods.

Our baseline analysis proved that no single feature-selection method was consistently optimal. For example, our 66-feature Pearson and Mutual Information filter worked best for the MLP model at a 3.56% MAPE, but degraded for Random Forest. Pure Permutation importance worked well for Random Forest but degraded the MLP, while SHAP selection biased toward tree models. This conflict confirmed that different models prefer different feature subsets, which strongly motivated our proposed Multi-Hybrid consensus framework.

Slide 20 — Multi-Hybrid Feature Selection Results

Here is the performance of the proposed Multi-Hybrid consensus framework. The Multi-Hybrid framework successfully compressed the active feature space by 72.73%, cutting the

feature count down from 66 to just 18 consensus features. Despite this massive reduction, the models maintained highly competitive forecasting performance. This heavily simplified our tuning costs and deployment complexity. Looking at our computational runtime metrics, this reduction made our exhaustive grid search feasible, running across 85,536 XGBoost combinations and 960 MLP hyperparameter pathways optimized on GPU and CPU devices.

Slide 21 — Ensemble Model Performance (2024 Holdout)

With our 18 consensus features locked in, here is how the complete stacking ensemble performed on our untouched 2024 test period. The proposed Meta-LR ensemble achieved the best overall performance, claiming a top MAPE of 3.61%, a low MAE of 76.40 megawatts, an RMSE of 108.50 megawatts, and a high R^2 score of 0.90. It outperformed every individual standalone base model, including SVR at 3.89%, MLP at 4.53%, XGBoost at 4.98%, and Random Forest at 6.25%. Notably, Meta-SVR failed to generalize, degrading to a 6.16% MAPE due to distribution variance in the out-of-fold inputs. These results fully validate the effectiveness of our 18-feature consensus framework.

Slide 22 — Monthly Forecast Performance

Beyond a single summary number, it's vital to see how that accuracy holds up across the full year. As we can see from the full-year 2024 chart, Meta-LR tracks the actual demand line with high precision throughout all twelve months. The model successfully captures Cambodia's highly seasonal demand variations—tracking the elevated peaks during the hot summer months and the drop during the cooler winter months—while maintaining a stable, consistent 3.61% MAPE across the entire test horizon.

Slide 23 — Hourly Resolution Detail

Zooming in to an hourly resolution highlights the true operational value of the ensemble layer. Looking closely at this detailed week in March 2024, the black dashed line represents the actual ground-truth load. The Meta-LR model tracks intraday peak demand and nightly troughs with exceptional accuracy. While individual base models exhibit visible underestimation or lag delays at peak hours, Meta-LR resolves these distribution errors, providing clear empirical evidence that our stacking architecture creates significant operational value.

Slide 24 — Discussion

Pulling all of these findings together into one cohesive view. Our discussion highlights four core pillars: first, our Consensus Feature Selection achieved a 72.73% reduction from 66 down to 18 features, lowering tuning complexity; second, feature insights confirmed that lag_24 and lag_168 dominate, while temperature stands out as the most powerful environmental predictor; third, our Meta-Stacking validation proved that Meta-LR dominates with a 3.61% MAPE and a 0.901 R^2 score; and fourth, the full system is verified and deployment-oriented, ready to support real grid operations.

Section 6 — Conclusion and Demonstration

Slide 25 — Outcomes

Coming back to where we started, let's check our progress against the four original objectives. Every core project objective has been fully achieved: our feature importance framework successfully mapped variable significance using four distinct methods; our group experiments validated G4 as the optimal 18-feature set; our modeling phase locked down a superior Meta-LR stacking engine at a 3.61% MAPE; and our technical engineering tasks delivered a production-ready React, FastAPI, and PostgreSQL platform with an 89% code coverage mark.

Slide 26 — SDG Impact

Beyond our technical achievements, this research delivers a measurable sustainable impact. This project contributes directly to Sustainable Development Goal 7: Affordable and Clean Energy. By providing grid operators with highly accurate load predictions, our system improves electricity grid reliability and operational efficiency. It enables a more sustainable energy supply framework across Cambodia and directly facilitates the safe integration of volatile renewable energy sources, such as solar power, into the national transmission grid.

Slide 27 — Challenges Overcome

Managing a project of this scale brought real technical challenges that required disciplined engineering solutions. We overcame four key roadblocks: first, resolving feature consensus when different importance lenses produced conflicting rankings; second, managing a rigorous full-stack workflow, bridging data science research with production systems; third, coordinating parallel workstreams to complete the ML engine and web app simultaneously; and fourth, optimizing limited compute resources, where complex hyperparameter search combinations demanded weeks of automated processing time.

Speaker note: State these challenges as professional engineering facts that you successfully managed. Keep your voice confident and objective.

Slide 28 — Lessons Learned

Working through those roadblocks provided invaluable personal and professional growth. I built deep technical capability in applied data science across the entire ML lifecycle, mastered full-stack MLOps containerization, and established strong engineering discipline by adhering to strict phase-gate verification rules rather than expedient shortcuts. Personally, this project expanded my self-learning capabilities, sharpened my root-cause problem-solving, and enhanced my communication skills, proving my ability to bridge the gap between complex research and full-scale engineering.

Slide 29 — Future Direction

Looking ahead, this platform establishes a foundation for five clear future directions. First, expanding our scope to multi-horizon forecasting, including 168-hour cycles for long-term

planning; second, integrating advanced deep learning base learners such as LSTM, Bi-LSTM, and Transformer models; third, deploying automated performance monitoring to flag MAPE degradation and trigger automatic retraining loops; fourth, building automated data acquisition directly linking EDC SCADA systems with the Open-Meteo API; and fifth, continuously upgrading platform features to enhance the operational user experience.

Platform Demonstration

Before I open the floor to your feedback, I would like to show you a brief demonstration of the operational platform. As we step into the Smart Load Forecasting Platform, we can see a secure, role-based application. Users can seamlessly navigate to the Warehouse page to upload fresh load and weather data files. The FastAPI backend automatically triggers our Pydantic gatekeepers to enforce strict schema consistency, preventing data mismatch errors. Moving to our Prediction module, users can select a 'Simulation Mode' to isolate past dates, execute our Meta-LR stacking engine in the background, render interactive actual-vs-predicted Recharts graphs, and download comprehensive forecast sheets instantly. This bridges the gap between machine learning research and real-world utility.

Closing Slide

That brings me to the end of my presentation. Thank you very much, respected Chairman, Reviewers, and Committee Members, for your valuable time and attention. I am now happy to take any questions, comments, or feedback you may have regarding my research. Thank you.